SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	**A.Sai Joshitha**	**Reg. No.:**	**192424220**
Date:		**Duration:**	60 minutes

Code of Conduct

I _____A.Sai Joshitha(192424220)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: ____A.Sai Joshitha_____

Annexure A – Git & Docker Technical Assignment

ONLINE MOVIE TICKET BOOKING SYSTEM

1. INTRODUCTION

The Online Movie Ticket Booking System is a web-based application designed to allow users to view available movies and book movie tickets through an online interface. The system provides a simple interaction between the user interface and backend application services.

The project is developed using a separate frontend and backend architecture. The frontend is responsible for displaying movie information and providing a booking interface to the user. The backend is responsible for providing REST API services for retrieving movie information and processing ticket booking requests.

Git is used as the version control system for maintaining source code and tracking changes made during development. Different Git branches are used to organize development activities and maintain a clean project history.

Docker is used for application containerization. The frontend and backend are containerized separately so that each component can be developed, deployed, and maintained independently. Docker Compose is used to define and run both containers as a multi-container application.

The major technologies used in the project are HTML, CSS, JavaScript, Python, Flask, Nginx, Git, Docker, and Docker Compose.

2. PROBLEM ANALYSIS

2.1 Problem Statement

Traditional movie ticket booking can require users to visit a theatre or use different platforms to check movie information and availability. An online movie ticket booking system provides a centralized interface through which users can view available movies and submit ticket booking requests.

The proposed system provides a simple web application where users can:

- View available movies.
- View movie genre and show time.
- Select a movie.
- Enter their name.
- Specify the number of tickets.
- Submit a booking request.
- Receive a booking confirmation message.

The application follows a frontend-backend architecture. The frontend communicates with the backend using REST API requests.

3. SYSTEM OBJECTIVES

The main objectives of the Online Movie Ticket Booking System are:

1. To develop a simple web-based movie ticket booking application.
2. To provide a user-friendly interface for viewing movies.
3. To allow users to select movies and specify the number of tickets.
4. To implement backend REST APIs using Python Flask.
5. To separate frontend and backend components.
6. To containerize the frontend and backend independently using Docker.
7. To use Docker Compose for running multiple containers.
8. To use Git for source-code version control.
9. To implement a suitable Git branching strategy.
10. To demonstrate a reproducible development and deployment environment.

4. FUNCTIONAL REQUIREMENTS

The major functional requirements are given below.

Requirement	**Description**
Movie Display	The system should display available movies.
Movie Information	The system should display movie title, genre, and show time.
Movie Selection	The user should be able to select a movie.
User Details	The user should enter their name.
Ticket Selection	The user should specify the number of tickets.
Booking	The system should process the booking request.
Confirmation	The system should display a booking confirmation message.
API Communication	Frontend should communicate with backend through REST APIs.

5. NON-FUNCTIONAL REQUIREMENTS

The system should satisfy the following non-functional requirements:

Performance

The application should respond quickly to user requests for movie information and booking operations.

Usability

The frontend should be simple and easy to understand.

Maintainability

The frontend and backend should be separated so that modifications to one component do not unnecessarily affect the other.

Portability

Docker containers should allow the application to run consistently on different development environments.

Scalability

The containerized architecture allows frontend and backend components to be scaled independently in a larger deployment.

Reliability

The application should provide predictable API responses and should be capable of being restarted through Docker Compose.

6. EXPECTED OUTCOMES

After completing the project, the expected outcomes are:

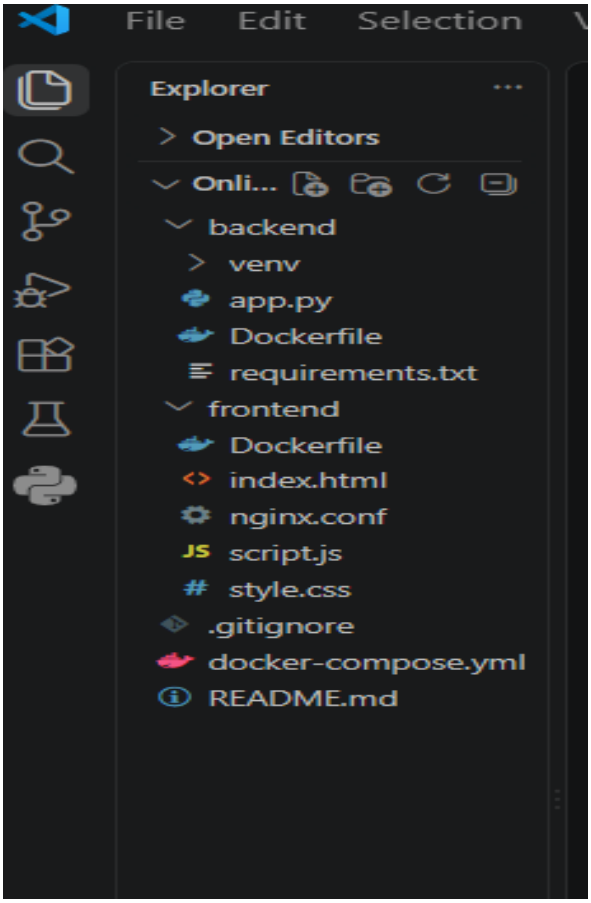
- A functional movie ticket booking web application.
- Separate frontend and backend source-code directories.
- A Git repository containing the complete source code.
- Meaningful Git commits.
- Multiple Git branches.
- Dockerfile for the frontend.
- Dockerfile for the backend.
- Docker Compose configuration.
- Separate frontend and backend Docker containers.
- Successful communication between frontend and backend.
- Successful movie retrieval through the API.
- Successful booking request processing.

7. PROJECT STRUCTURE DESIGN

The project repository is organized as follows:

Online-Movie-Ticket-Booking-System/

```
│
├── frontend/
│   ├── index.html
│   ├── style.css
│   ├── script.js
│   ├── Dockerfile
│   └── nginx.conf
│
├── backend/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── docker-compose.yml
├── .gitignore
└── README.md
```



7.1 Frontend Folder

The frontend folder contains all files required for the user interface.

index.html

This file defines the structure of the movie booking webpage. It contains the movie listing section and booking form.

style.css

This file contains the styling rules used to improve the appearance of the application.

script.js

This file contains JavaScript logic. It communicates with the Flask backend through HTTP API requests and processes the booking response.

Dockerfile

The frontend Dockerfile defines how the frontend application is packaged into an Nginx-based Docker image.

nginx.conf

This configuration file controls how Nginx serves the frontend application.

7.2 Backend Folder

The backend folder contains the server-side application.

app.py

This is the main Flask application. It provides API endpoints such as:

GET /movies

POST /book

requirements.txt

This file lists Python packages required by the backend application.

Dockerfile

This file defines the Docker image required to run the Flask backend.

7.3 Root Files

docker-compose.yml

This file defines the frontend and backend services and their port mappings.

.gitignore

This file specifies files and folders that should not be tracked by Git.

README.md

This file provides project documentation, installation instructions, and usage information.

8. SOFTWARE ARCHITECTURE

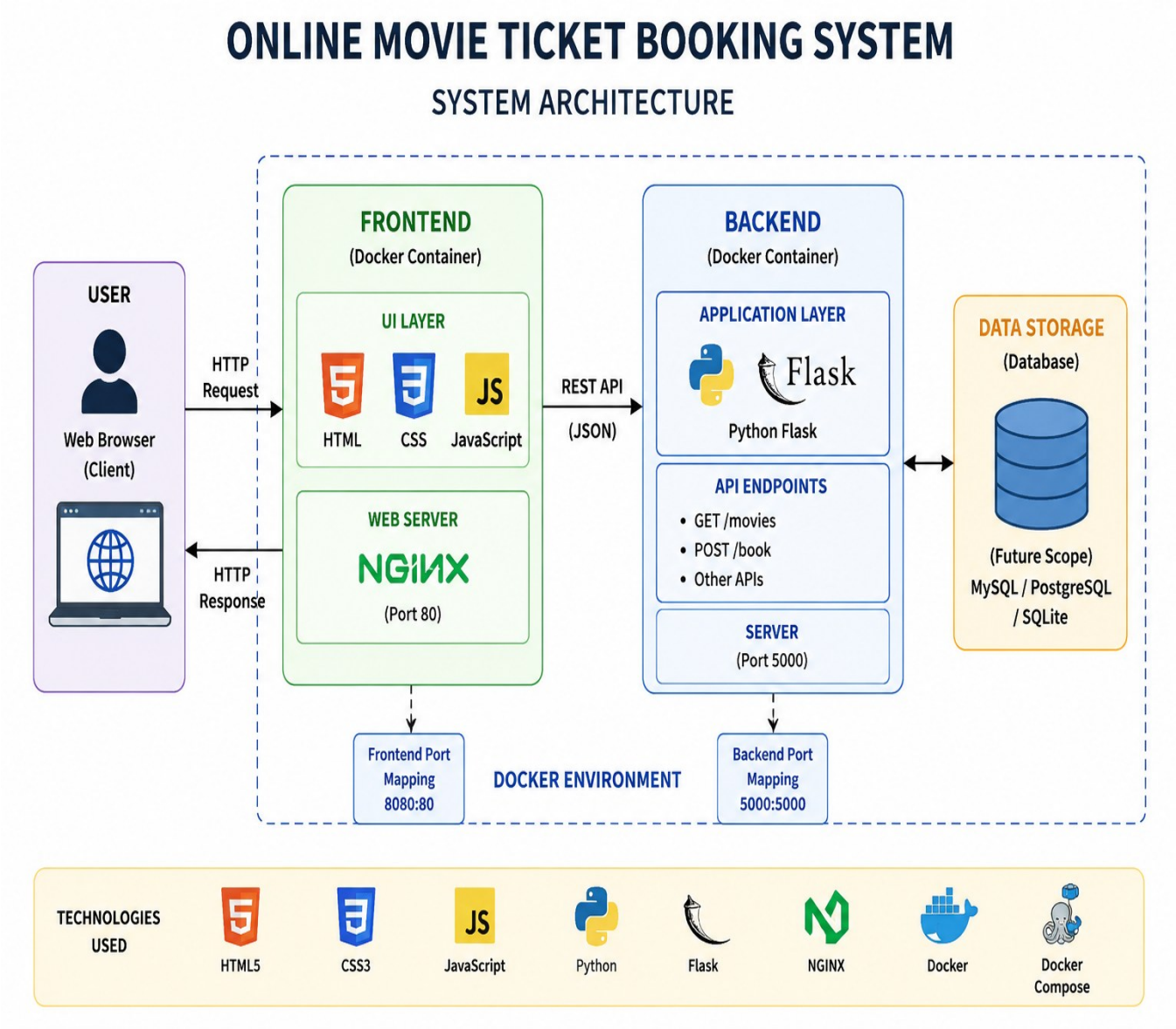
The Online Movie Ticket Booking System follows a simple layered client-server architecture.

The system consists of:

1. User
2. Frontend
3. Backend
4. Docker containers

The user interacts with the frontend through a web browser. The frontend sends API requests to the Flask backend. The backend processes the request and sends the response back to the frontend.

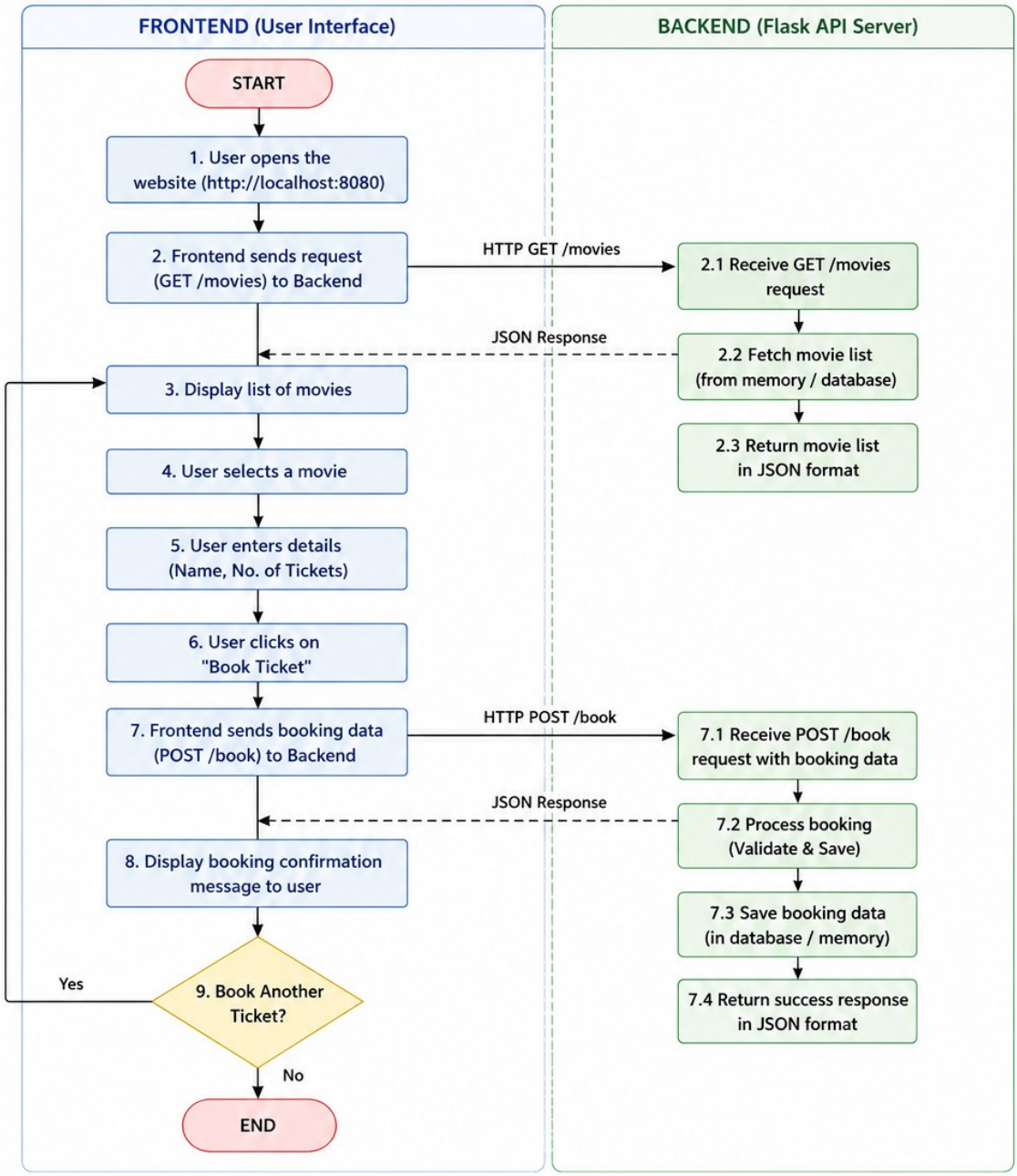
8.1 System Architecture



9. COMPONENT INTERACTION

ONLINE MOVIE TICKET BOOKING SYSTEM

FLOW DIAGRAM



LEGEND

- Start / End
- Frontend Process
- Backend Process
- Decision
- Request / Flow
- Response

FLOW DESCRIPTION

1. User opens the website.
2. Frontend requests movie list from Backend.
3. Backend returns movie list.
4. User selects a movie and enters details.
5. Frontend sends booking data to Backend.
6. Backend processes the booking and saves it.
7. Backend returns success response.
8. Frontend displays confirmation to the user.
9. User can choose to book another ticket or end the process.

/

v

10. GIT REPOSITORY INITIALIZATION

Git is used to maintain the project source code and track changes.

10.1 Initialize Repository

The repository is initialized using:

git init

This creates a hidden .git directory that stores Git metadata, including information about commits, branches, and repository configuration.

10.2 Check Repository Status

The repository status can be checked using:

git status

This command displays untracked, modified, and staged files.

10.3 Add Project Files

The project files are added to the staging area using:

git add .

10.4 Create Initial Commit

The first commit is created using:

git commit -m "Initial project setup"

The commit records the initial version of the project.

11. IMPORTANT GIT FILES

.git Folder

The .git directory is created automatically by git init. It stores Git's internal repository information.

.gitignore

The .gitignore file prevents unnecessary files from being committed.

Example:

__pycache__/

*.pyc

venv/

.venv/

.env

.vscode/

The virtual environment and temporary Python files should not be stored in the Git repository.

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git init
Initialized empty Git repository in C:/Users/jjosh/OneDrive/Desktop/Online-Movie-Ticket-Booking-System/.git/
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        backend/
        docker-compose.yml
        frontend/

nothing added to commit but untracked files present (use "git add" to track)
```

12. GIT BRANCHING STRATEGY

A branching strategy is used to organize development work.

The following branches are used:

main

|

+---- development

|

+---- feature/frontend

|

+---- feature/backend

Main Branch

The main branch contains stable versions of the application.

Development Branch

The development branch is used to integrate features before they are considered ready for the stable version.

Feature Branches

Feature branches are created for independent development activities.

Examples:

feature/frontend

feature/backend

This approach allows frontend and backend work to be developed independently.

13. BRANCH CREATION COMMANDS

Create the development branch:

git branch development

Switch to development:

git switch development

Create frontend feature branch:

git switch -c feature/frontend

Create backend feature branch:

git switch -c feature/backend

List branches:

git branch

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git add .
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git commit -m "Initial project setup"
[master (root-commit) 0c3ee78] Initial project setup
 11 files changed, 321 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 backend/Dockerfile
 create mode 100644 backend/app.py
 create mode 100644 backend/requirements.txt
 create mode 100644 docker-compose.yml
 create mode 100644 frontend/Dockerfile
 create mode 100644 frontend/index.html
 create mode 100644 frontend/nginx.conf
 create mode 100644 frontend/script.js
 create mode 100644 frontend/style.css
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git status
On branch master
nothing to commit, working tree clean
```

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch development
Switched to branch 'development'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git merge feature/frontend
Already up to date.
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git merge feature/backend
Already up to date.
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git log --oneline --graph --all
* 0c3ee78 (HEAD -> development, master, feature/frontend, feature/backend) Initial project setup
```

14. VERSION CONTROL WORKFLOW

The Git workflow used in this project consists of the following stages:

Working Directory

|

| git add

v

Staging Area

|

| git commit

v

Local Repository

|

| git push

v

Remote Repository

14.1 Add Changes

git add .

14.2 Commit Changes

git commit -m "Implement movie booking frontend"

14.3 Backend Commit

git add backend

git commit -m "Implement movie booking backend"

14.4 Merge Feature Branch

Switch to development:

git switch development

Merge frontend:

git merge feature/frontend

Merge backend:

git merge feature/backend

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git branch
  * master
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch -c development
Switched to a new branch 'development'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch -c feature/frontend
Switched to a new branch 'feature/frontend'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch -c feature/backend
Switched to a new branch 'feature/backend'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git branch
    development
  * feature/backend
    feature/frontend
    master
```

15. REPOSITORY SYNCHRONIZATION

If a remote GitHub repository is created, the local repository can be synchronized using:

git remote add origin <repository-url>

The main branch can be pushed using:

git push -u origin main

The development branch can be pushed using:

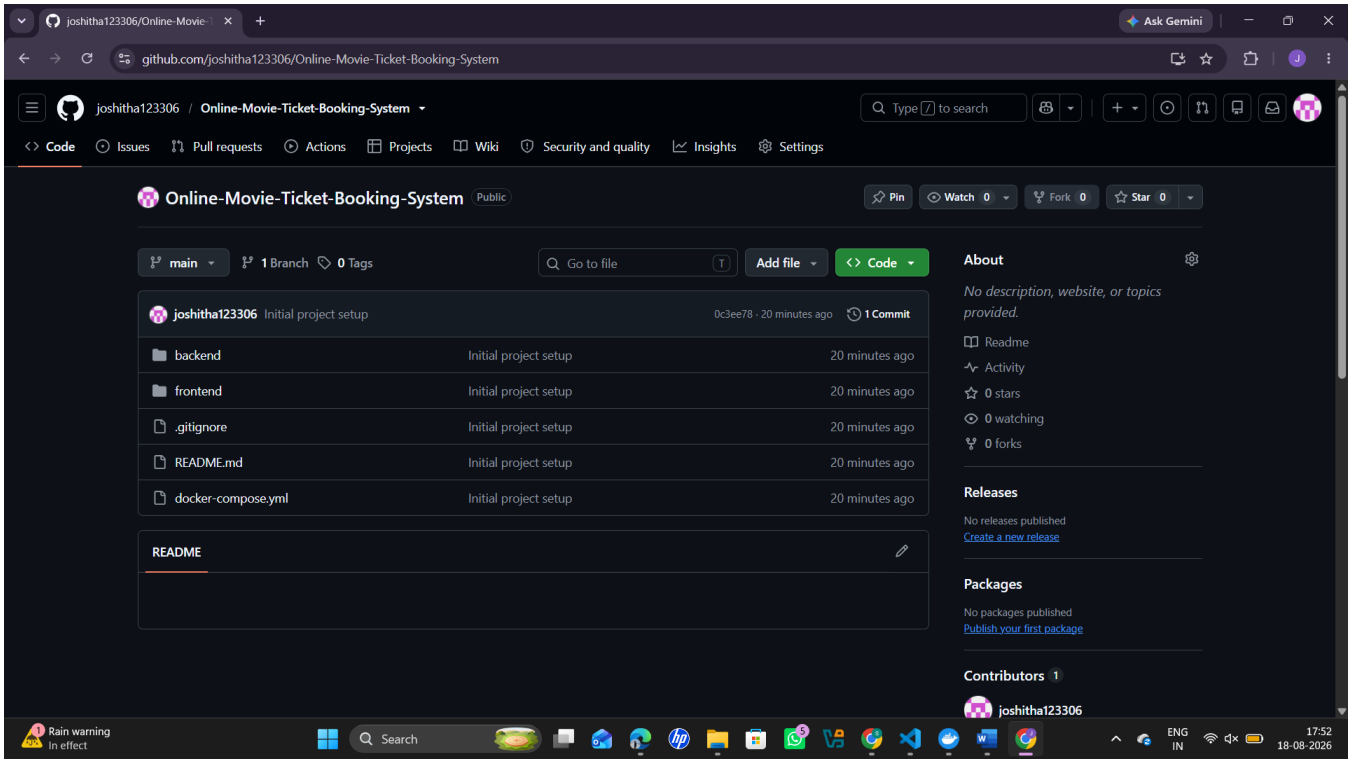
git push -u origin development

Feature branches can also be pushed when required.

Synchronization allows the source code to be stored remotely and provides a backup of the project.

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git remote add origin https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System.git
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git remote -v
origin  https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System.git (fetch)
origin  https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System.git (push)
```

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git remote -v
origin  https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System.git (fetch)
origin  https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System.git (push)
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git push -u origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (15/15), 3.16 KiB | 462.00 KiB/s, done.
Total 15 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/joshitha123306/Online-Movie-Ticket-Booking-System.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```



16. COMMIT HISTORY ANALYSIS

Commit history can be viewed using:

git log --oneline

A graphical view can be obtained using:

git log --oneline --graph --all

Meaningful commit messages are used in the project.

Examples:

Initial project setup

Implement movie booking frontend

Implement movie booking backend

Add Docker configuration

Configure Docker Compose

Fix frontend API communication

Good commit messages make it easier to understand what changed in each version.

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch feature/frontend
Switched to branch 'feature/frontend'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git status
On branch feature/frontend
nothing to commit, working tree clean
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git add frontend
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git commit -m "Implement movie booking frontend"
On branch feature/frontend
nothing to commit, working tree clean
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git log --oneline --all
0c3ee78 (HEAD -> feature/frontend, master, feature/backend, development) Initial project setup
```

```
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git switch feature/backend
Switched to branch 'feature/backend'
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git status
On branch feature/backend
nothing to commit, working tree clean
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git add backend
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git commit -m "Implement movie booking backend"
On branch feature/backend
nothing to commit, working tree clean
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> git log --oneline --all
0c3ee78 (HEAD -> feature/backend, master, feature/frontend, development) Initial project setup
```

17. DOCKER ENVIRONMENT DESIGN

Docker is used to package the application and its dependencies into isolated containers.

The project uses two separate containers:

Frontend Container

The frontend is deployed using Nginx.

Backend Container

The backend is deployed using Python Flask.

The two containers are managed using Docker Compose.

18. FRONTEND DOCKERFILE

The frontend Dockerfile is:

FROM nginx:alpine

COPY . /usr/share/nginx/html

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Explanation

FROM nginx:alpine

Uses a lightweight Nginx image.

COPY

Copies the frontend files into the Nginx web directory.

nginx.conf

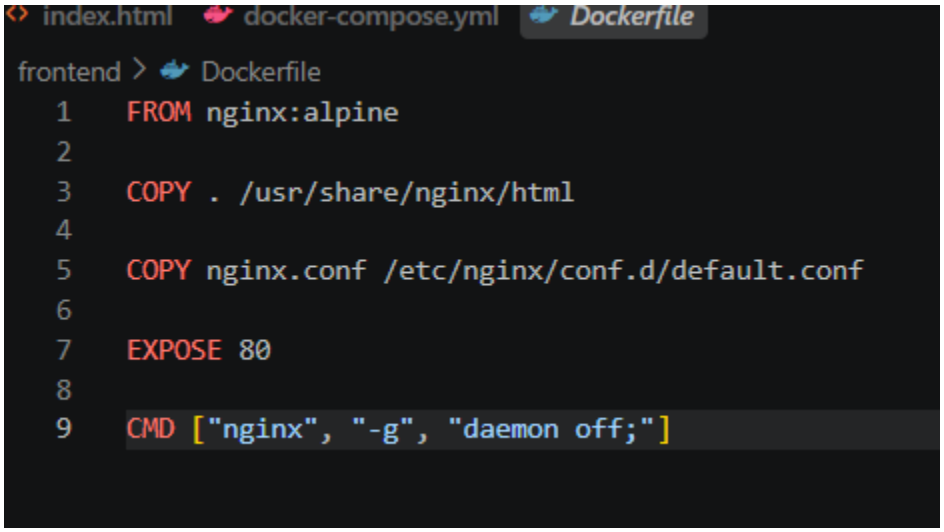
Copies the custom Nginx configuration.

EXPOSE 80

Specifies the port used by Nginx inside the container.

CMD

Starts the Nginx web server.



19. BACKEND DOCKERFILE

The backend Dockerfile is:

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]

Explanation

FROM

Selects Python as the base environment.

WORKDIR

Creates the application working directory.

COPY

Copies dependency and source files.

RUN

Installs the Python dependencies.

EXPOSE

Specifies the Flask application port.

CMD

Starts the Flask application.

backend > Dockerfile

```
1   FROM python:3.11-slim
2
3   WORKDIR /app
4
5   COPY requirements.txt .
6
7   RUN pip install --no-cache-dir -r requirements.txt
8
9   COPY app.py .
10
11  EXPOSE 5000
12
13  CMD ["python", "app.py"]
```

20. DOCKER COMPOSE DESIGN

Docker Compose is used to manage the frontend and backend containers as a single application.

The configuration is:

services:

frontend:

build:

context: ./frontend

container_name: movie_frontend

ports:

- "8080:80"

depends_on:

- backend

backend:

build:

context: ./backend

container_name: movie_backend

ports:

- "5000:5000"

Frontend Service

The frontend service builds the Dockerfile inside the frontend folder.

The port mapping is:

8080:80

This means:

Host Port 8080 → Container Port 80

Therefore, the application can be accessed from:

http://localhost:8080

Backend Service

The backend service builds the Dockerfile inside the backend folder.

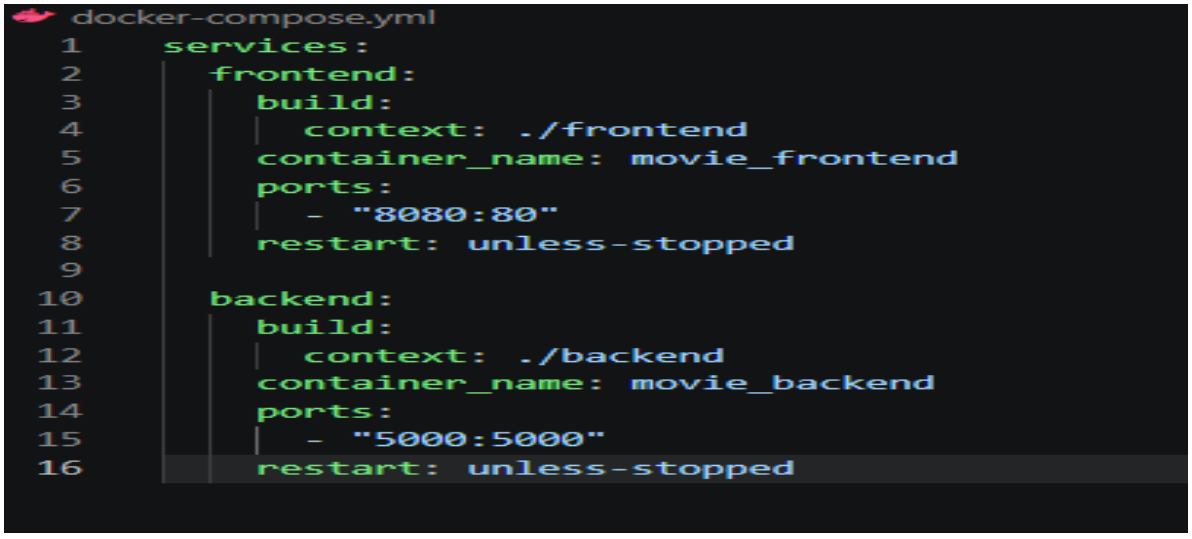
The port mapping is:

5000:5000

Therefore, the Flask API can be accessed through port 5000.

depends_on

The depends_on configuration specifies that the frontend service depends on the backend service.



21. DOCKER IMAGE BUILDING

Before running the containers, Docker images are built using:

docker compose build

This command reads the Docker Compose configuration and builds the frontend and backend images.

The frontend image is created from:

frontend/Dockerfile

The backend image is created from:

backend/Dockerfile

22. CONTAINER DEPLOYMENT

The complete application is started using:

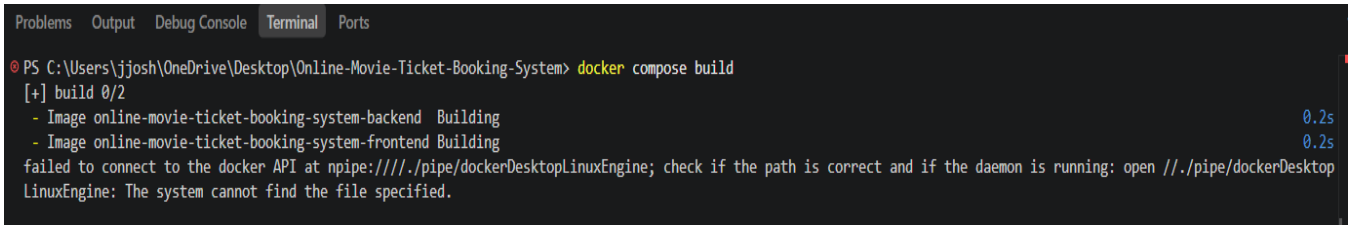
docker compose up

For rebuilding and starting together:

docker compose up --build

The command starts both services defined in the Compose file.

The frontend container runs Nginx and the backend container runs Flask.



23. VERIFYING RUNNING CONTAINERS

The running containers are checked using:

docker ps

or:

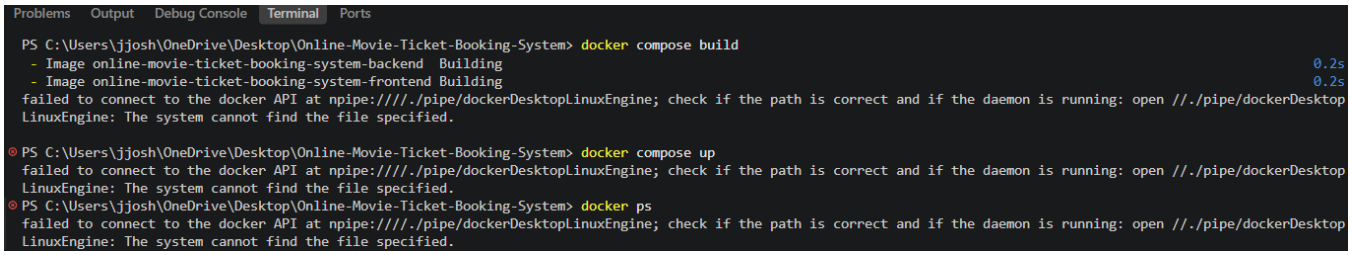
docker compose ps

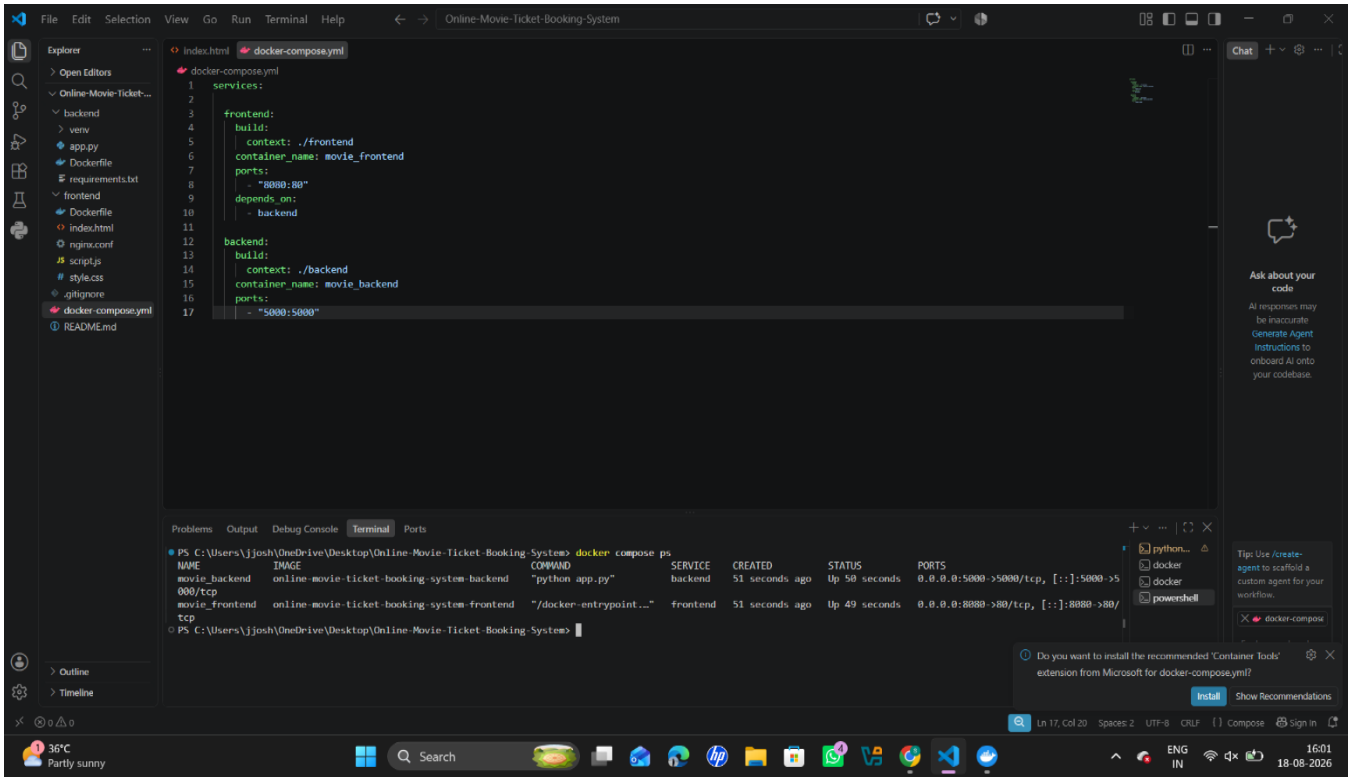
The expected result is approximately:

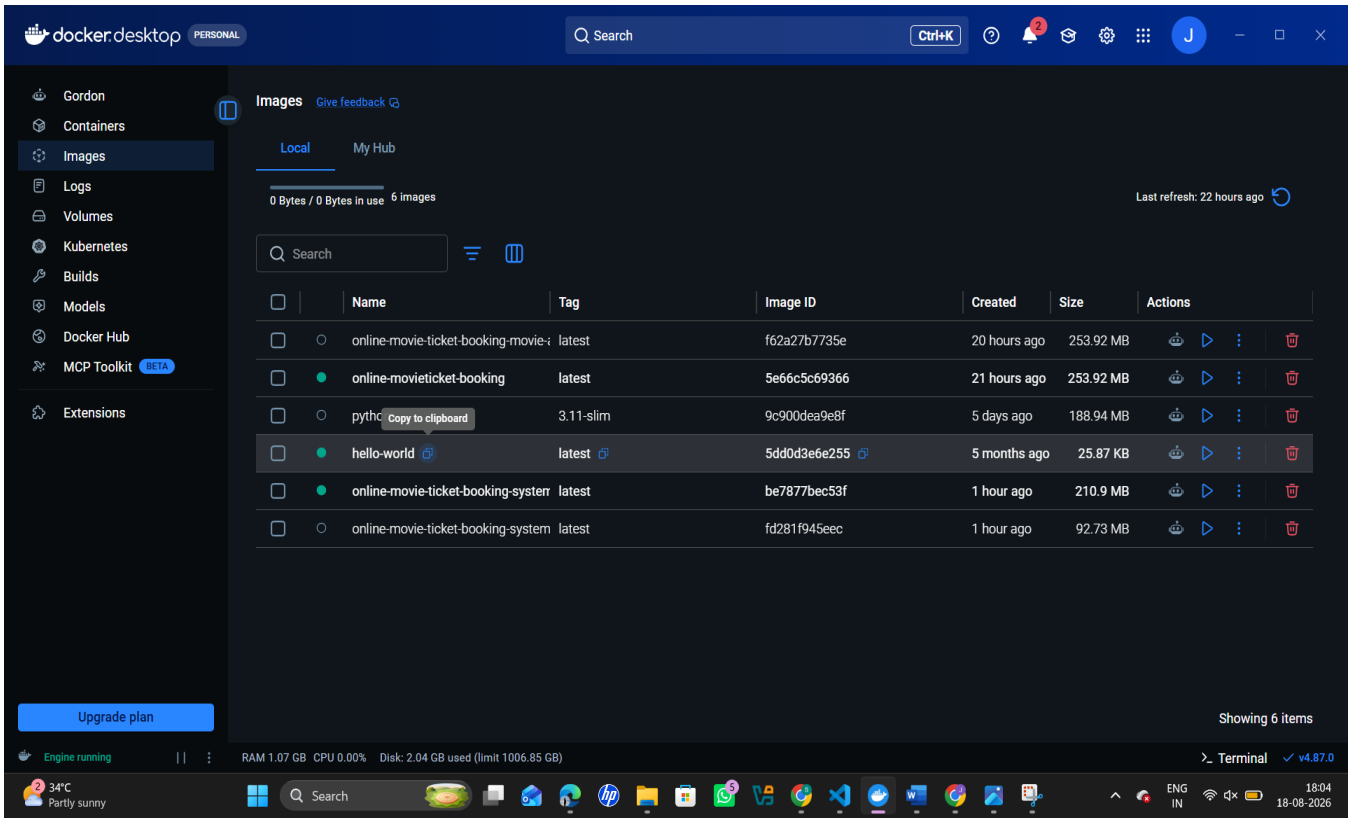
movie_frontend Up 0.0.0.0:8080->80/tcp

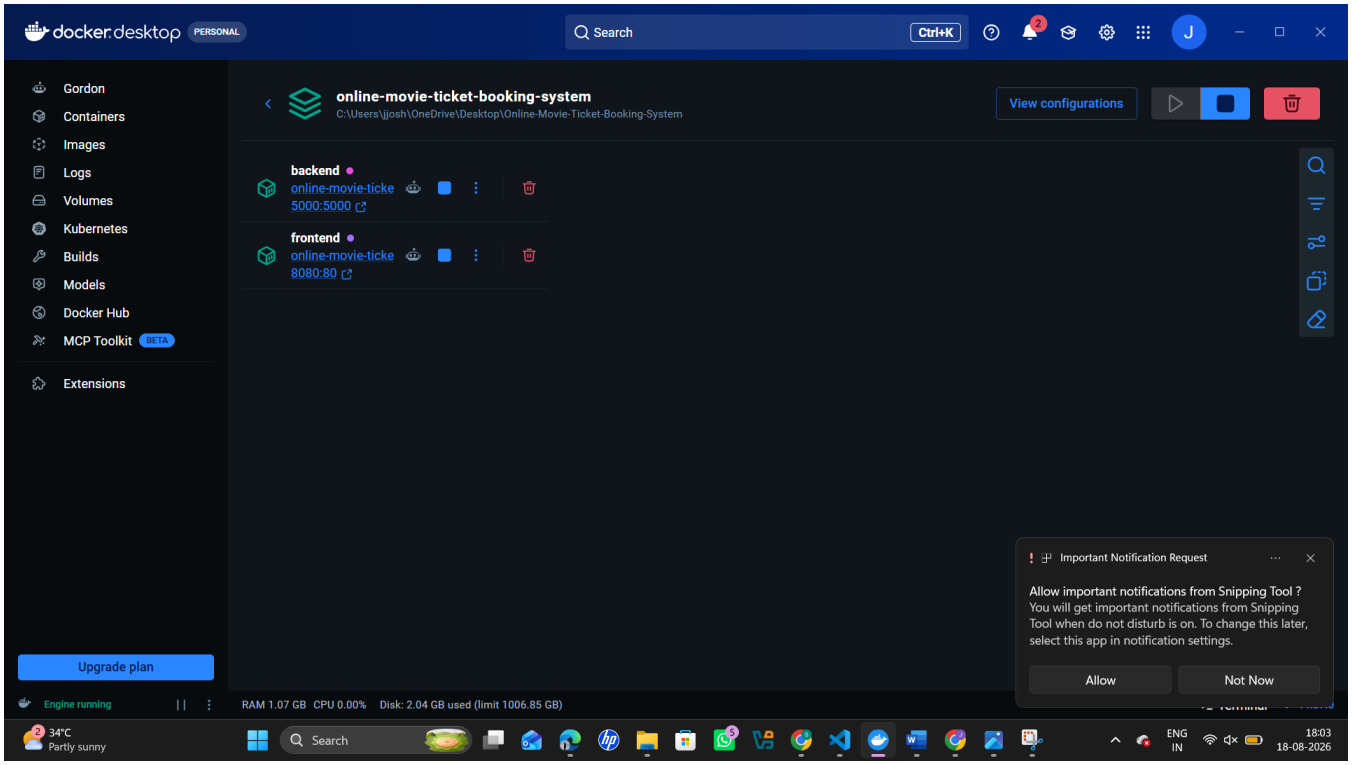
movie_backend Up 0.0.0.0:5000->5000/tcp

The port mappings confirm that the containers are accessible from the host system.




File Edit Selection View Go Run Terminal Help
Online-Movie-Ticket-Booking-System
Explorer
Open Editors
Online-Movie-Ticket-...
backend
venv
app.py
Dockerfile
requirements.txt
frontend
Dockerfile
index.html
nginx.conf
script.js
style.css
.gitignore
docker-compose.yml
README.md
index.html
docker-compose.yml
docker-compose.yml
services:
frontend:
build:
context: ./frontend
container_name: movie_frontend
ports:
- "8080:80"
depends_on:
- backend
backend:
build:
context: ./backend
container_name: movie_backend
ports:
- "5000:5000"
Chat
Ask about your code
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems Output Debug Console Terminal Ports
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
movie_backend online-movie-ticket-booking-system-backend "python app.py" backend 51 seconds ago Up 50 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
movie_frontend online-movie-ticket-booking-system-frontend "/docker-entrypoint..." frontend 51 seconds ago Up 49 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\jjosh\OneDrive\Desktop\Online-Movie-Ticket-Booking-System>
python...
docker
docker
powershell
Tip: Use /create-agent to scaffold a custom agent for your workflow.
docker-compose
Do you want to install the recommended 'Container Tools' extension from Microsoft for docker-compose.yml?
Install
Show Recommendations
Outline
Timeline
Ln 17, Col 20 Spaces 2 UTF-8 CRLF Compose Sign in
36°C
Partly sunny
Search
ENG
IN
16:01
18-08-2026


docker desktop
PERSONAL
Search
Ctrl+K
Gordon
Containers
Images
Logs
Volumes
Kubernetes
Builds
Models
Docker Hub
MCP Toolkit
BETA
Extensions
Upgrade plan
Images
Give feedback
Local
My Hub
0 Bytes / 0 Bytes in use
6 images
Last refresh: 22 hours ago
Search
Name
Tag
Image ID
Created
Size
Actions
online-movie-ticket-booking-movie-
latest
f62a27b7735e
20 hours ago
253.92 MB
online-movieticket-booking
latest
5e66c5c69366
21 hours ago
253.92 MB
pytho
Copy to clipboard
3.11-slim
9c900dea9e8f
5 days ago
188.94 MB
hello-world
latest
5dd0d3e6e255
5 months ago
25.87 KB
online-movie-ticket-booking-systen
latest
be7877bec53f
1 hour ago
210.9 MB
online-movie-ticket-booking-system
latest
fd281f945eec
1 hour ago
92.73 MB
Showing 6 items
Engine running
RAM 1.07 GB CPU 0.00% Disk: 2.04 GB used (limit 1006.85 GB)
Terminal
v4.87.0
34°C
Partly sunny
Search
ENG
IN
18:04
18-08-2026



24. APPLICATION TESTING

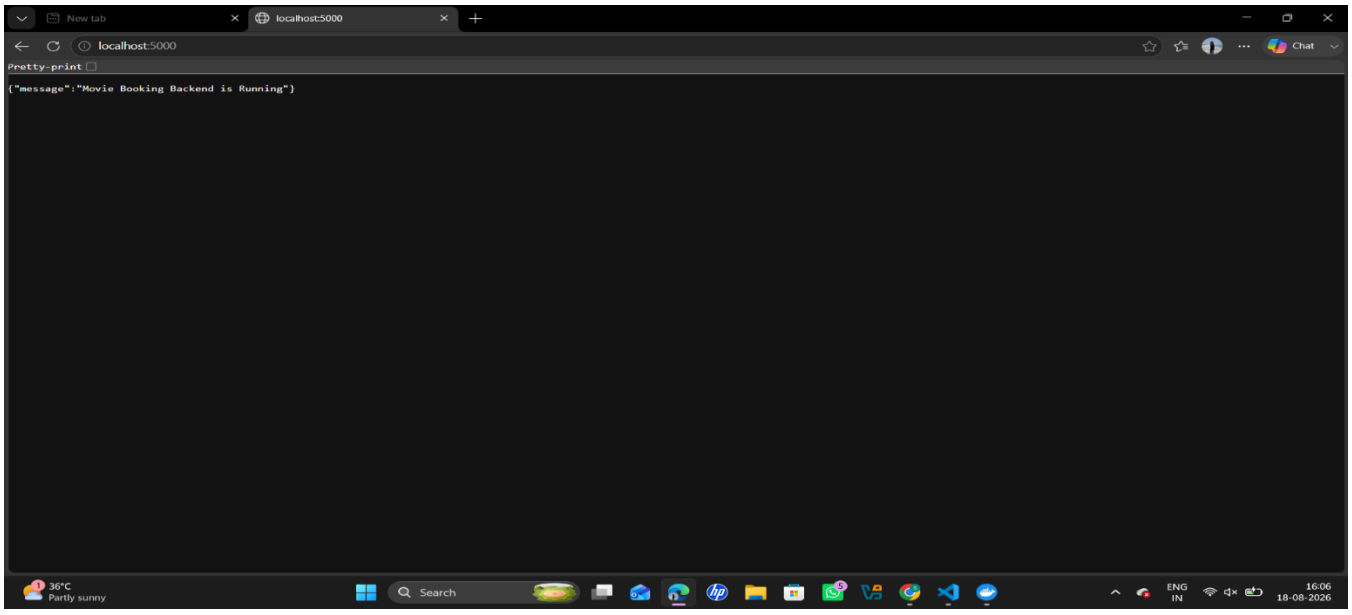
The application is tested in multiple stages.

Test 1 — Backend Test

Open:

http://localhost:5000

The backend should return a message indicating that the movie booking backend is running.

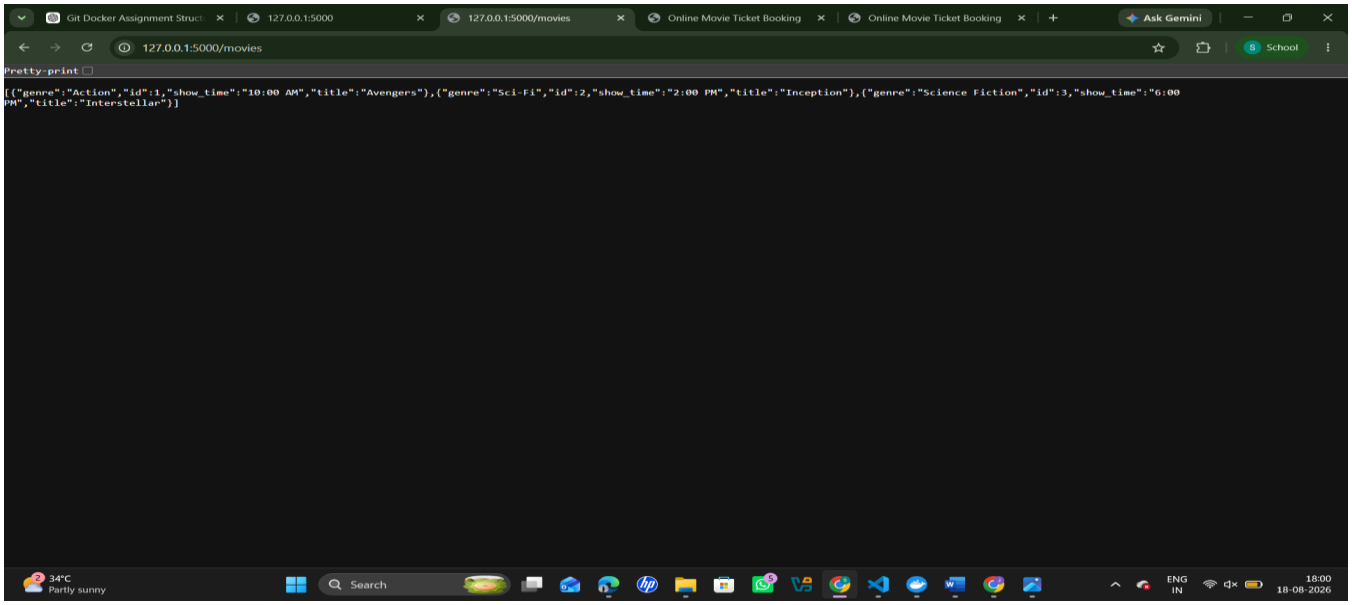


Test 2 — Movie API Test

Open:

http://localhost:5000/movies

The backend should return the available movie information.

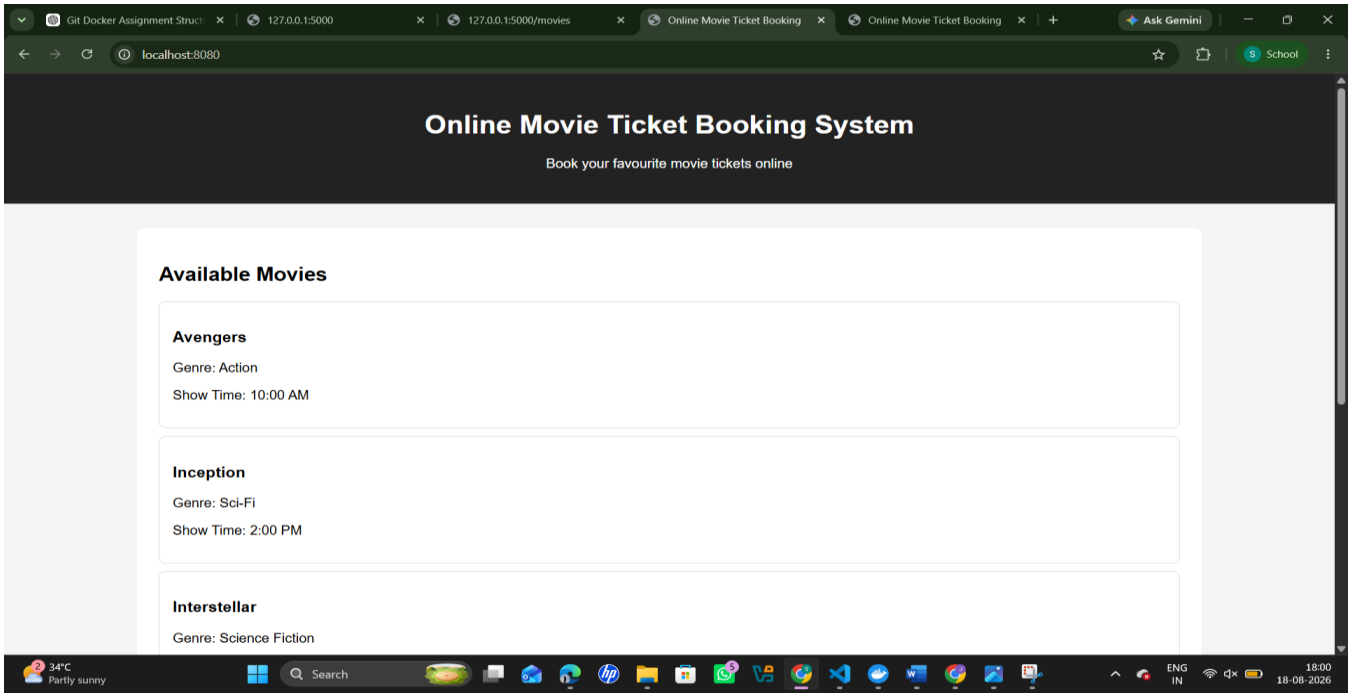


Test 3 — Frontend Test

Open:

http://localhost:8080

The movie booking webpage should be displayed.



Test 4 — Booking Test

Enter:

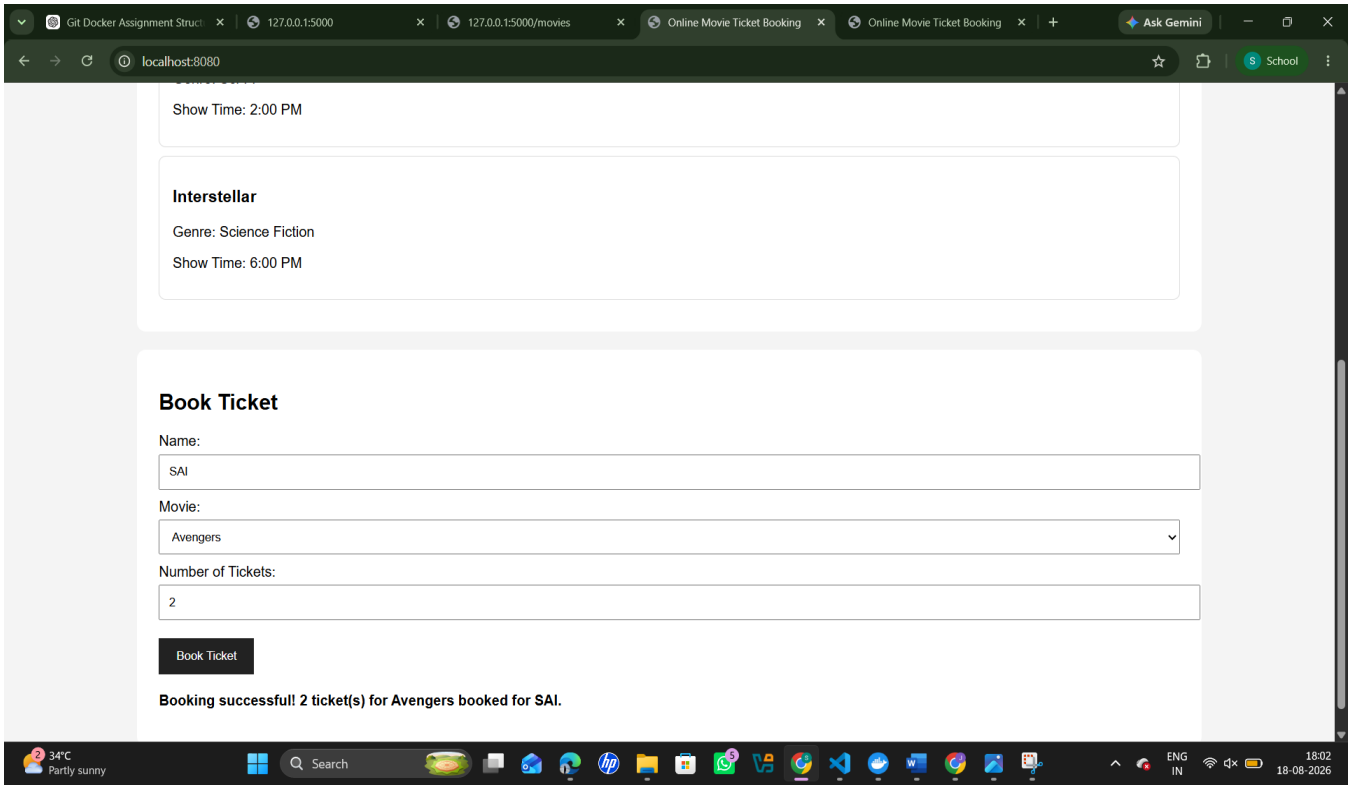
Name: Sai

Movie: Avengers

Number of Tickets: 2

Click the booking button.

The application should display a booking confirmation.



25. FRONTEND-BACKEND COMMUNICATION

The frontend communicates with the backend through HTTP REST API calls.

For retrieving movies, JavaScript sends:

GET /movies

The backend returns movie information.

For booking a ticket, the frontend sends:

POST /book

with booking information.

The overall communication is:

Browser

|

v

Frontend :8080

|

| GET /movies

| POST /book

v

Backend :5000

|

v

Response

|

v

Frontend

|

v

User

26. DOCKER NETWORKING

Docker Compose creates an application network for the services.

The services can communicate with each other through the Compose network.

The main advantages are:

- Service isolation.
- Easy communication between containers.
- Simplified deployment.
- Independent container management.
- Consistent networking configuration.

The host accesses the frontend through port 8080 and the backend through port 5000.

27. BENEFITS OF GIT

Git provides several advantages for this project.

Version Control

Git records changes made to source code.

Collaboration

Different developers can work on separate branches.

Branch Management

Frontend and backend development can be separated using feature branches.

Rollback

Earlier versions can be restored when necessary.

Change Tracking

Commit history provides a record of project modifications.

Remote Backup

GitHub can be used as a remote repository.

28. BENEFITS OF DOCKER

Portability

The application can run consistently across different systems.

Isolation

Frontend and backend dependencies are isolated.

Reproducibility

The same Docker configuration can recreate the application environment.

Easy Deployment

The application can be started using Docker Compose.

Maintainability

Each application component can be maintained independently.

Scalability

Containers can be replicated and managed independently in larger deployments.

29. COMBINED BENEFITS OF GIT AND DOCKER

Git and Docker complement each other.

Git manages the source-code lifecycle while Docker manages the application runtime environment.

Git

|

+-- Source Code

+-- Version History

+-- Branches

+-- Collaboration

|

v

Docker

|

+-- Application Images

+-- Containers

+-- Dependencies

+-- Deployment

Together they provide a reliable software development and deployment workflow.

30. CHALLENGES ENCOUNTERED

During implementation, several challenges can occur.

30.1 Docker Engine Not Running

Docker commands can fail when Docker Desktop is not running.

For example, Docker may display an error indicating that it cannot connect to the Docker API.

The solution is to start Docker Desktop and verify the Docker Engine status.

30.2 Port Mapping Issues

The frontend container uses port 80 internally and maps it to port 8080 on the host.

The correct configuration is:

```
ports:
  - "8080:80"
```

The backend uses:

```
ports:
  - "5000:5000"
```

30.3 Frontend-Backend Communication

The frontend needs to communicate with the Flask backend through an HTTP API.

CORS configuration is used in the Flask backend to allow browser-based requests.

30.4 Dependency Management

The Python backend requires Flask and Flask-CORS.

These dependencies are maintained in:

requirements.txt

30.5 Container Debugging

Docker logs can be used to identify application errors.

Commands include:

docker compose logs

and:

docker compose logs backend

SCREENSHOT 18 — DOCKER LOGS

Insert a screenshot showing Docker logs.

Use:

docker compose logs

Caption:

Figure 18: Docker container logs used for application monitoring and debugging

31. FUTURE ENHANCEMENTS

The current system can be improved in several ways.

Database Integration

A database such as MySQL or PostgreSQL can be added to permanently store:

- Users
- Movies
- Theatres
- Show timings
- Seat availability
- Bookings
- Payment information

Authentication

User registration and login can be added.

Seat Selection

Users can select specific seats from an interactive seat layout.

Payment Gateway

A secure payment gateway can be integrated.

Booking History

Users can view their previous bookings.

Admin Dashboard

An administrator can manage movies, theatres, schedules, and bookings.

Three-Tier Docker Architecture

The application can be expanded to:

Frontend Container

|

v

Backend Container

|

v

Database Container

This would provide a more realistic production-style architecture.

CI/CD

GitHub Actions or another CI/CD platform can automatically build and test Docker images whenever code is pushed.

32. ARCHITECTURAL QUALITY ATTRIBUTES

Scalability

The frontend and backend can be scaled independently.

Maintainability

The separation between frontend and backend makes code maintenance easier.

Portability

Docker provides a consistent execution environment.

Availability

Containers can be restarted when failures occur.

Security

Future versions can implement HTTPS, authentication, secure environment variables, and database security.

Performance

Nginx provides efficient static file serving while Flask handles backend API operations.

3. FINAL TESTING TABLE

Test Case	Input/Action	Expected Result	Status
TC01	Start backend	Flask server starts	Pass
TC02	Open /	Backend response displayed	Pass
TC03	Open /movies	Movie information displayed	Pass
TC04	Start Docker Compose	Containers start	Pass
TC05	Open port 8080	Frontend displayed	Pass
TC06	Select movie	Movie selected	Pass
TC07	Enter ticket count	Ticket count accepted	Pass
TC08	Submit booking	Confirmation displayed	Pass

34. CONCLUSION

The Online Movie Ticket Booking System demonstrates the practical application of Git version control and Docker containerization concepts. The project is organized into separate frontend and backend components, making the architecture easier to understand, develop, test, and maintain. Git provides version tracking through repositories, branches, commits, and merging, while Docker provides isolated and reproducible environments for the frontend and backend. Docker Compose simplifies the management of multiple containers and allows the application components to be started together. The implementation demonstrates how modern software engineering practices can be applied to a web-based application. Future improvements such as database integration, authentication, seat selection, online payment, CI/CD, and a three-tier container architecture can make the system more suitable for real-world deployment.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25